

Code:

```
.global _start
_start:
    # TODO_1:
    # 1. Load the trap_entry address into a register
    # 2. Write it into the control status register (CSR), mtvec using the csrw instruction
    la t0, trap_entry
    csrw mtvec, t0 ##Machine Trap Vector (stores the base address for handling CPU exceptions and
interrupts (traps) in Machine mode)

    # init stack pointer
    la sp, __stack_start # points to the start of the stack, see linker script.

    # TODO_2:
    # 1. Display a message in the terminal before triggering the trap handler
    # 2. (Optional) Display the current program counter value
    li t3, 0x10000000
    la t1, before
    addi t2, t1, 20
before_loop:
    lb t4, 0(t1)
    beqz t4, before_done    # branch if = 0
    sb t4, 0(t3)
    addi t1, t1, 1          #increment t1 by 1 till it reaches 20
    blt t1, t2, before_loop #branch if < (so in this case it will loop till t1 < t2)

    ## Optional TODO here
    auipc t0, 0
    # Print 0
    li t4, '0'
    sb t4, 0(t3)
    # Print x
    li t4, 'x'
    sb t4, 0(t3)
    li t6, 16
hex_bfr_loop:
    srli t4, t0, 60
    andi t4, t4, 0xF
    li t2, 10
    bltu t4, t2, is_bfr_digit
    addi t4, t4, ('a' - 10)    # 10..15 -> 'a'..'f'
    j emit_bfr
is_bfr_digit:
    addi t4, t4, '0'          # 0..9 -> '0'..'9'
emit_bfr:
    sb t4, 0(t3)
    slli t0, t0, 4
    addi t6, t6, -1
    bnez t6, hex_bfr_loop
    li t4, '\n'
    sb t4, 0(t3)

before_done:
    # TODO_3:
    # 1. Trigger a software interrupt with the ecall instruction
    ecall

    # TODO_4:
    # 1. Display a message in the terminal after coming back from the trap handler
    # 2. (Optional) Display the current program counter value
    # 3. Halt the program using a loop
    la t1, after
    addi t2, t1, 20
after_loop:
    lb t4, 0(t1)
    sb t4, 0(t3)
    addi t1, t1, 1
```

```

        blt t1, t2, after_loop
        auipc t0, 0
        ## Optional TODO here
        # Print 0
        li t4, '0'
        sb t4, 0(t3)
        # Print x
        li t4, 'x'
        sb t4, 0(t3)
        li t6, 16
hex_afr_loop:
        srli t4, t0, 60
        andi t4, t4, 0xF
        li t2, 10
        bltu t4, t2, is_afr_digit
        addi t4, t4, ('a' - 10)    # 10..15 -> 'a'..'f'
        j emit_afr
is_afr_digit:
        addi t4, t4, '0'          # 0..9 -> '0'..'9'
emit_afr:
        sb t4, 0(t3)             # output one hex char
        slli t0, t0, 4
        addi t6, t6, -1
        bnez t6, hex_afr_loop
        li t4, '\n'
        sb t4, 0(t3)
forever:
        j forever

handle_trap:
        # TODO_5:
        # 1. Display a message in the terminal that tells the program is inside the trap handler
        # 2. (Optional) Display the current program counter value
        li t3, 0x10000000
        la t1, inside
        addi t2, t1, 20
inside_loop:
        lb t4, 0(t1)
        beqz t4, inside_done      # branch if = 0
        sb t4, 0(t3)
        addi t1, t1, 1
        blt t1, t2, inside_loop
inside_done:
        # TODO_6:
        # 1. Load the return address from the mepc reg
        # 2. Increment the address by one instruction
        # 3. Save the updated address in the mepc re
        # 4. Jump back to the trap_entry
        csrr t0, mepc ##Machine Exception Program Counter register return address
        addi t1, t0, 4
        csw mepc, t1
        ret

.align 2
trap_entry:
        # store state before handling the trap
        sd ra, 0*8(sp)
        sd a0, 1*8(sp)
        sd a1, 2*8(sp)
        sd a2, 3*8(sp)
        sd a3, 4*8(sp)
        sd a4, 5*8(sp)
        sd a5, 6*8(sp)
        sd a6, 7*8(sp)
        sd a7, 8*8(sp)
        sd t0, 9*8(sp)
        sd t1, 10*8(sp)
        sd t2, 11*8(sp)
        sd t3, 12*8(sp)

```

```

sd t4, 13*8(sp)
sd t5, 14*8(sp)
sd t6, 15*8(sp)
addi sp, sp, 17*8

# TODO_7:
# 1. Jump to the handle_trap
jal ra, handle_trap          #j handle_trap
##https://pages.hmc.edu/harris/ddca/ddcarv/DDCARv\_AppB\_Harris.pdf

```

```

# load state after handling the trap
addi sp, sp, -17*8
ld ra, 0*8(sp)
ld a0, 1*8(sp)
ld a1, 2*8(sp)
ld a2, 3*8(sp)
ld a3, 4*8(sp)
ld a4, 5*8(sp)
ld a5, 6*8(sp)
ld a6, 7*8(sp)
ld a7, 8*8(sp)
ld t0, 9*8(sp)
ld t1, 10*8(sp)
ld t2, 11*8(sp)
ld t3, 12*8(sp)
ld t4, 13*8(sp)
ld t5, 14*8(sp)
ld t6, 15*8(sp)

```

```

# TODO_8:
# 1. Return to the main code using the mret instruction
mret

```

```

.section .data

```

```

before:
    .string "Before trap handler\n"

```

```

inside:
    .string "Inside trap handler\n"

```

```

after:
    .string "After trap handler\n"

```

Observations and Insights:

link.ld suggests that the memory address starts from 0x80000000 for the stack.

. = ALIGN(8);

Because the linker script has a +8 byte of padding/alignment provided the “_start” begins at 0x80000008.

The PC is also pointing to the next instruction at memory address: 0x80000008.

```
Reading symbols from trap.elf...
(No debugging symbols found in trap.elf)
(gdb) b _start
Breakpoint 1 at 0x80000008
(gdb) b before_loop
Breakpoint 2 at 0x80000024
(gdb) b hex_bfr_loop
Breakpoint 3 at 0x80000050
(gdb) b after_loop
Breakpoint 4 at 0x80000094
(gdb) b hex_afr_loop
Breakpoint 5 at 0x800000bc
(gdb) b inside_loop
Breakpoint 6 at 0x80000104
(gdb) c
The program is not being run.
(gdb) target remote :12222
Remote debugging using :12222
0x0000000000000100 in ?? ()
```

The initial state of registers before entering the assembly code.

```
Remote debugging using :12222
0x0000000000001000 in ?? ()
(gdb) info registers
ra          0x0      0x0
sp          0x0      0x0
gp          0x0      0x0
tp          0x0      0x0
t0          0x0      0
t1          0x0      0
t2          0x0      0
fp          0x0      0x0
s1          0x0      0
a0          0x0      0
a1          0x0      0
a2          0x0      0
a3          0x0      0
a4          0x0      0
a5          0x0      0
a6          0x0      0
a7          0x0      0
s2          0x0      0
s3          0x0      0
s4          0x0      0
s5          0x0      0
s6          0x0      0
s7          0x0      0
s8          0x0      0
s9          0x0      0
s10         0x0      0
s11         0x0      0
t3          0x0      0
t4          0x0      0
t5          0x0      0
t6          0x0      0
pc          0x1000   0x1000
(gdb) c
Continuing.
```

```

Breakpoint 1, 0x0000000800000008 in _start ()
(gdb) info registers
ra          0x0          0x0
sp          0x0          0x0
gp          0x0          0x0
tp          0x0          0x0
t0          0x80000128    2147483944
t1          0x0          0
t2          0x0          0
fp          0x0          0x0
s1          0x0          0
a0          0x0          0
a1          0x87e00000    2279604224
a2          0x1028       4136
a3          0x0          0
a4          0x0          0
a5          0x0          0
a6          0x0          0
a7          0x0          0
s2          0x0          0
s3          0x0          0
s4          0x0          0
s5          0x0          0
s6          0x0          0
s7          0x0          0
s8          0x0          0
s9          0x0          0
s10         0x0          0
s11         0x0          0
t3          0x0          0
t4          0x0          0
t5          0x0          0
t6          0x0          0
pc          0x80000008    0x80000008 <_start+8>
(gdb) c
Continuing.

```

The t0 is currently at 0x80000128 because we have loaded the address of mtvec into t0.

```

(gdb) info registers mepc
mepc          0x80000084    2147483780
Missing register name
(gdb) info registers mtvec
mtvec         0x80000128    2147483944

```

Session after entering the before_loop. pc register address is updated to the next instruction.

```
Breakpoint 2, 0x0000000080000024 in before_loop ()
(gdb) info registers
ra          0x0          0x0
sp          0x800001f8      0x800001f8
gp          0x0          0x0
tp          0x0          0x0
t0          0x80000128      2147483944
t1          0x800001b8      2147484088
t2          0x800001cc      2147484108
fp          0x0          0x0
s1          0x0          0
a0          0x0          0
a1          0x87e00000      2279604224
a2          0x1028         4136
a3          0x0          0
a4          0x0          0
a5          0x0          0
a6          0x0          0
a7          0x0          0
s2          0x0          0
s3          0x0          0
s4          0x0          0
s5          0x0          0
s6          0x0          0
s7          0x0          0
s8          0x0          0
s9          0x0          0
s10         0x0          0
s11         0x0          0
t3          0x10000000      268435456
t4          0x0          0
t5          0x0          0
t6          0x0          0
pc          0x80000024      0x80000024 <before_loop>
(gdb) c
Continuing.
```

la t1, before

- After loading the address of the first character of the .before into the t1 register.

addi t2, t1, 20

- The add increment will enable the t1 register to work towards the address of t2.

addi t1, t1, 1

- The add increment will increment the address of t1 register by 1.

blt t1, t2, inside_loop

- blt is an instruction which compares the address of the t1 and t2 register. The inside_loop will keep running until the $t1 < t2$ condition is satisfied.

```
[akawadga@bondi ~]$ qemu-system-riscv64 -machine virt -nographic -bios none -kernel trap.elf -S -gdb tcp::12222
Before
```

The memory address of the t0, t1 and pc is updated after the before_loop is executed successfully.

The pc is currently pointing to the next instruction's address, since each character is looping through the t1 register and being updated at each cycle.

```
qemu-system-riscv64 terminating on signal 12 from pid 2221222 (core dumped)
[akawadga@bondi ~]$ qemu-system-riscv64 -machine virt -nographic -bios none -kernel trap.elf -S -gdb tcp::12222
Before trap handler
0x0000000080000038
Inside trap handler
```

```
Breakpoint 2, 0x0000000080000024 in before_loop ()
(gdb) c
Continuing.

Breakpoint 3, 0x0000000080000050 in hex_bfr_loop ()
(gdb) info registers
ra                0x0          0x0
sp                0x800001f8      0x800001f8
gp                0x0          0x0
tp                0x0          0x0
t0                0x80000038      2147483704
t1                0x800001cc      2147484108
t2                0x800001cc      2147484108
fp                0x0          0x0
s1                0x0          0
a0                0x0          0
a1                0x87e00000      2279604224
a2                0x1028         4136
a3                0x0          0
a4                0x0          0
a5                0x0          0
a6                0x0          0
a7                0x0          0
s2                0x0          0
s3                0x0          0
s4                0x0          0
s5                0x0          0
s6                0x0          0
s7                0x0          0
s8                0x0          0
s9                0x0          0
s10               0x0          0
s11               0x0          0
t3                0x10000000      268435456
t4                0x78          120
t5                0x0          0
t6                0x10          16
pc                0x80000050      0x80000050 <hex_bfr_loop>
```

The .inside's and after's string was printed using the same logic as the one utilized for .before .

```
[akawadga@bondi ~]$ qemu-system-riscv64 -machine virt -nographic -bios none -kernel trap.elf -S -gdb tcp::12222
Before trap handler
0x0000000080000038
Inside trap handler
After trap handler
█
```

```
Breakpoint 3, 0x0000000080000094 in after_loop ()
(gdb) info registers
ra          0x0          0x0
sp          0x800001f8    0x800001f8
gp          0x0          0x0
tp          0x0          0x0
t0          0x0          0
t1          0x800001f5    2147484149
t2          0x800001f6    2147484150
fp          0x0          0x0
s1          0x0          0
a0          0x0          0
a1          0x87e00000    2279604224
a2          0x1028       4136
a3          0x0          0
a4          0x0          0
a5          0x0          0
a6          0x0          0
a7          0x0          0
s2          0x0          0
s3          0x0          0
s4          0x0          0
s5          0x0          0
s6          0x0          0
s7          0x0          0
s8          0x0          0
s9          0x0          0
s10         0x0          0
s11         0x0          0
t3          0x10000000    268435456
t4          0xa         10
t5          0x0          0
t6          0x0          0
pc          0x80000094    0x80000094 <after_loop>
```

```
[akawadga@bondi ~]$ qemu-system-riscv64 -machine virt -nographic -bios none -kernel trap.elf -S -gdb tcp::12222
Before trap handler
0x0000000080000038
Inside trap handler
After trap handler
0x00000000800000a4
█
```

```
(gdb) c
Continuing.
^C
Program received signal SIGINT, Interrupt.
0x00000000800000f0 in forever ()
(gdb) info registers
ra            0x0          0x0
sp            0x800001f8    0x800001f8
gp            0x0          0x0
tp            0x0          0x0
t0            0x0          0
t1            0x800001f6    2147484150
t2            0xa         10
fp            0x0          0x0
s1            0x0          0
a0            0x0          0
a1            0x87e00000    2279604224
a2            0x1028       4136
a3            0x0          0
a4            0x0          0
a5            0x0          0
a6            0x0          0
a7            0x0          0
s2            0x0          0
s3            0x0          0
s4            0x0          0
s5            0x0          0
s6            0x0          0
s7            0x0          0
s8            0x0          0
s9            0x0          0
s10           0x0          0
s11           0x0          0
t3            0x10000000    268435456
t4            0xa         10
t5            0x0          0
t6            0x0          0
pc            0x800000f0    0x800000f0 <forever>
```

Kill the debug session to free up the emulator's memory.

```
[akawadga@bondi ~]$ ps aux | grep qemu-system-riscv64
shenglioc 1001415 50.9 0.0 1704728 50076 ? S1 Jan19 4922:51 qemu-system-riscv64 -M virt -bios none -serial stdio -display none -kernel hello.img -s -S
chenangl 1421138 0.0 0.0 1704804 49264 ? S1 Jan22 0:04 qemu-system-riscv64 -M virt -bios none -serial stdio -display none -kernel trap.img -gdb tcp::23234 -S
hongduyp 1608630 0.0 0.0 1704804 49644 ? S1 Jan23 0:05 qemu-system-riscv64 -M virt -bios none -serial stdio -display none -kernel trap.img -gdb tcp::5986 -S
chenangl 1622019 0.0 0.0 1704736 50160 ? S1 Jan23 0:05 qemu-system-riscv64 -M virt -bios none -serial stdio -display none -kernel hello.img -gdb tcp::18342 -S
cmeny 1623293 0.0 0.0 1704728 50844 ? S1 Jan23 0:05 qemu-system-riscv64 -M virt -bios none -serial stdio -display none -kernel program.img -gdb tcp::18356 -S
wenkins 2038494 97.5 0.0 1704804 50776 ? S1 Jan25 1247:42 qemu-system-riscv64 -M virt -bios none -serial stdio -display none -kernel trap.img -gdb tcp::51280 -S
akawadga 2161357 2.9 0.0 1702620 43840 pts/2 S1+ 15:35 0:26 qemu-system-riscv64 -machine virt -nographic -bios none -kernel trap.elf -S -gdb tcp::12222
melindmt 2164361 0.0 0.0 12808 2924 pts/5 S+ 15:48 0:00 /bin/sh -c qemu-system-riscv64 -M virt -bios none -serial stdio -display none -kernel trap.img -gdb tcp::<YOUR PORT NUMBER> -S & riscv6
4-unknown-elf-gdb trap.elf -tui
akawadga 2164683 0.0 0.0 16480 1268 pts/1 S+ 15:49 0:00 grep --color=auto qemu-system-riscv64
```

```
[akawadga@bondi ~]$ kill 2161357
[akawadga@bondi ~]$
```

FINAL OUTPUT:

```
[akawadga@bondi ~]$ qemu-system-riscv64 -machine virt -nographic -bios none -kernel trap.elf -S -gdb tcp::12222
Before trap handler
0x0000000080000038
Inside trap handler
After trap handler
0x00000000800000a4
qemu-system-riscv64: terminating on signal 15 from pid 2129439 (-bin/tcsh)
[akawadga@bondi ~]$
```